

Module Name: auxwall-hr

HR Document Tracking Sub-Module (Backend) Documentation

This document outlines the complete specifications, database schema, API endpoints, and development workflow for the HR Document Tracking Sub-Module. This module will manage all company and staff-related documentation, including expiry tracking, categorization, and activity logging.

1. Module Overview and Goals

The primary goal is to provide a robust, scalable backend solution for centralizing all organizational documents, ensuring easy retrieval, and managing expiry dates with automatic reminders.

- **Technology Stack:** Node.js / Express.js (or similar)
- **Database ORM:** Sequelize (using the main application's passed connection).
- **Multi-Tenancy:** All records must be associated with a `companyId`.
- **Prefixing:** All database entities and API endpoints must start with `hr_`.

2. Development Workflow (For Staff)

This workflow defines the steps for building and integrating the sub-module.

Stage	Task	Owner	Status
0. Setup	Set up local development environment (Node, Git, IDE).	Developer	☒
1. Database	Define and create <i>local testing</i> database schema (<code>hr_ tables</code>).	Developer	☒
2. Core API	Implement CRUD APIs for <code>hr_category</code> and <code>hr_document</code> .	Developer	☐

3. Logic API	Implement Dashboard, Expiry, and Activity APIs.	Developer	☐
4. Integration	Update local environment to use passed sequelize instance and main module API access point.	Developer	☐
5. Testing	Unit testing, API testing (Postman/Insomnia), and functional testing.	QA/Developer	☐

3. Database Schema (Sequelize Models)

The module requires four primary tables. All tables must include the **companyId** foreign key.

3.1. **hr_categories** (Document Categories)

Field Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique Category ID.
companyId	UUID/INTEGER	NOT NULL, INDEX	Foreign Key to the main module's Company table.

hr_category_name	STRING	NOT NULL	e.g., "Staff Documents", "Tax Filings", "Vehicle Registration".
hr_category_parent_id	INTEGER	NULL, Foreign Key to hr_categories.id	Used for hierarchy (e.g., "Staff Documents" is parent of "Staff Name").
createdBy	INTEGER	staff.id	
createdAt, updatedAt	DATE	NOT NULL	Standard Sequelize timestamps.

3.2. hr_documents (Core Document Storage)

Field Name	Data Type	Constraints	Description	
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique Document ID.	
companyId	INTEGER	NOT NULL, INDEX	Foreign Key to the Company table.	
hr_category_id	INTEGER	NOT NULL, Foreign Key to hr_categories.id	The category this document belongs to.	

hr_document_name	STRING	NOT NULL	The user-friendly name of the document.
hr_file_path	STRING	NOT NULL	The path/name used for file storage (S3 Key, Local Path, etc.).
hr_file_size_bytes	INTEGER	NULL	Size of the stored file.
hr_mime_type	STRING	NULL	e.g., <i>application/pdf</i> , <i>image/jpeg</i> .
hr_staff_id	UUID/INTEGER	NULL, INDEX	Foreign Key to the main module's Staff table (if <i>hr_is_staff_specific</i> is true).
hr_expiry_date	DATEONLY	NULL	The date the document expires.
hr_reminder_days	INTEGER	NULL	Number of days before expiry to trigger a reminder (e.g., 30, 60).
hr_status	ENUM('Active', 'Expired', 'Archived')	DEFAULT 'Active'	Document status.

hr_uploaded_by_id	INTEGER	NOT NULL	Foreign Key to the user who uploaded the document.
createdAt, updatedAt	DATE	NOT NULL	Standard Sequelize timestamps.

3.3. hr_activities (Activity Logging)

Field Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique Activity ID.
companyId	INTEGER	NOT NULL, INDEX	Foreign Key to the Company table.
hr_document_id	INTEGER	NOT NULL, Foreign Key to hr_documents.id	The document the activity is related to.
hr_user_id	INTEGER	NOT NULL, Foreign key to staffs	The user who performed the action.
hr_action_type	ENUM('Upload', 'Update', 'Delete', 'Download', 'View')	NOT NULL	The type of action performed.

hr_description	TEXT	NULL	A brief detail of the change (e.g., "Updated Expiry Date")
createdAt	DATE	NOT NULL	Time the activity occurred.

4. API Endpoints Specification

All API endpoints will be prepended by the main module's base API path (e.g., [/api/hr/](#)).

4.1. Core CRUD Endpoints

Method	Endpoint	Description	Query Parameters (For all GET list endpoints)
GET	/hr_categories	Get list of all categories.	companyId, page, limit, hr_category_parent_id
POST	/hr_categories	Create a new category.	(Body: companyId, hr_category_name, etc.)
PUT	/hr_categories/:id	Update category details.	(Body: fields to update)
DELETE	/hr_categories/:id	Delete category.	(Deletes all associated documents - use caution).
GET	/hr_documents	Get list of all documents (paginated).	companyId, page, limit, hr_category_id, hr_staff_id, hr_status, search

GET	/hr_documents/:id	Get specific document details.	N/A
POST	/hr_documents	Upload a new document.	(Body: All fields + File/URL from Frontend/Main Module)
PUT	/hr_documents/:id	Update document metadata (e.g., expiry date, name).	(Body: fields to update)
DELETE	/hr_documents/:id	Delete document and remove file from storage.	N/A

4.2. Dashboard and Logic Endpoints

These endpoints provide aggregated data for the module's dashboard.

Method	Endpoint	Description
GET	/hr_dashboard	Retrieve all dashboard metrics in one call.
GET	/hr_dashboard/categories/summary	Total number of categories and the count of documents in each category.
GET	/hr_dashboard/expiry/near	Near Expiry List (Latest 15 documents expiring soon). Sorted by <code>hr_expiry_date</code> ascending. Filter: <code>hr_expiry_date > TODAY</code> and <code>hr_expiry_date < (TODAY + 60 days)</code> .

GET	/hr_dashboard/activities/recent	Recent Activities List (Latest 15 actions). Sorted by <i>createdAt</i> descending
GET	/hr_documents/search/categorized	Search documents with advanced categorization and filtering (e.g., search by name, filter by category and staff).

5. File Storage and API Integration

Storage location - `hr/companyid/[documentid]_[createddate]`

- **File Handling:** The actual file upload (multipart form data) should be handled by the **Main Module's API** or the **Frontend** which then saves the file to the shared file storage.
- **Module Responsibility:** This HR module's API (`POST /hr_documents`) receives the metadata and the final `hr_file_path` (Key/URL) from the caller and records this path in the database.
- **File Retrieval:** The `GET /hr_documents/:id` endpoint returns the `hr_file_path`. The frontend will use this path to directly fetch the file from the storage system (if public) or request the Main Module's file access API to retrieve it.

6. Reminders and Expiry Logic

- **Reminder Calculation:** The near expiry list is based on documents where `hr_expiry_date` is not null and is within a specific window (e.g., 60 days from the current date).
- **Cron Job (Main Module):** A scheduled task (Cron Job) must be set up in the **Main Module** (or a separate worker service) to:
 1. Daily or hourly, query all documents in the `hr_documents` table.
 2. Identify documents where `hr_expiry_date - TODAY <= hr_reminder_days`.
 3. Trigger notifications (Email, In-App, etc.) to the relevant HR staff based on the `companyid`.

7. Local Development Setup

To allow independent development, use a local Sequelize configuration before integrating with the main module.

1. **Local Database:** Set up a local PostgreSQL/MySQL database named `hr_document_test`.

Environment Variables:

Bash

Local Development Database

HR_DB_NAME=hr_document_test

HR_DB_USER=root

HR_DB_PASSWORD=

HR_DB_HOST=localhost

Local Development API Port

HR_API_PORT=3001

HR_API_PREFIX=/test/hr

2.

3. **Local Sequelize Configuration:** Use the local environment variables to establish the connection and run migrations before integration.
4. **Mocking:** Use mock data for `companyId` and `hr_staff_id` during local testing.

Would you like me to refine the dashboard endpoints or provide the complete Sequelize model definitions for the backend staff?